

Improving SIMD Parallelism via Dynamic Binary Translation

DING-YONG HONG, Institute of Information Science, Academia Sinica

YU-PING LIU and SHENG-YU FU, Department of Computer Science and Information Engineering,
National Taiwan University

JAN-JAN WU, Institute of Information Science, Academia Sinica

WEI-CHUNG HSU, Department of Computer Science and Information Engineering,
National Taiwan University

Recent trends in SIMD architecture have tended toward longer vector lengths, and more enhanced SIMD features have been introduced in newer vector instruction sets. However, legacy or proprietary applications compiled with short-SIMD ISA cannot benefit from the long-SIMD architecture that supports improved parallelism and enhanced vector primitives, resulting in only a small fraction of potential peak performance. This article presents a dynamic binary translation technique that enables short-SIMD binaries to exploit benefits of new SIMD architectures by rewriting short-SIMD loop code. We propose a general approach that translates loops consisting of short-SIMD instructions to machine-independent IR, conducts SIMD loop transformation/optimization at this IR level, and finally translates to long-SIMD instructions. Two solutions are presented to enforce SIMD load/store alignment, one for the problem caused by the binary translator's internal translation condition and one general approach using dynamic loop peeling optimization. Benchmark results show that average speedups of 1.51× and 2.48× are achieved for an ARM NEON to x86 AVX2 and x86 AVX-512 loop transformation, respectively.

CCS Concepts: • **General and reference** → **Design; Performance**; • **Computer systems organization** → **Single instruction, multiple data**; • **Software and its engineering** → **Dynamic compilers**;

Additional Key Words and Phrases: Dynamic binary translation, SIMD, vectorization, dynamic loop peeling, compiler annotation

ACM Reference format:

Ding-Yong Hong, Yu-Ping Liu, Sheng-Yu Fu, Jan-Jan Wu, and Wei-Chung Hsu. 2018. Improving SIMD Parallelism via Dynamic Binary Translation. *ACM Trans. Embed. Comput. Syst.* 17, 3, Article 61 (February 2018), 27 pages.

<https://doi.org/10.1145/3173456>

This work is supported in part by Ministry of Science and Technology of Taiwan under grant number MOST-106-2218-E-002-040.

Authors' addresses: D.-Y. Hong, Institute of Information Science, Academia Sinica, 128 Academia Road, Section 2, Nankang, Taipei, 115, Taiwan; email: dyhong@iis.sinica.edu.tw; Y.-P. Liu, Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan; email: r04922005@csie.ntu.edu.tw; S.-Y. Fu, Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan; email: d03922013@csie.ntu.edu.tw; J.-J. Wu, Institute of Information Science, Academia Sinica, Taipei, Taiwan; email: wuj@iis.sinica.edu.tw; W.-C. Hsu, Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan; email: hsuwc@csie.ntu.edu.tw.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 ACM 1539-9087/2018/02-ART61 \$15.00

<https://doi.org/10.1145/3173456>

1 INTRODUCTION

Dynamic Binary Translation (DBT) is a common technique to execute an application binary of one guest ISA on a host machine with a different ISA. It operates directly on binaries, providing the powerful functionality to inspect and modify runtime behavior of applications without source code available. DBTs have been widely used in many fields, such as runtime optimization (Bala et al. 2000; Wang et al. 2007), program analysis (Luk et al. 2005; Nethercote and Seward 2007), computer security (Kiriansky et al. 2002), and hardware design (Yourst 2007). One important application of DBT is that it allows for cross-ISA migration of legacy or proprietary applications. For example, many Android applications are either compiled for the ARM ISA or include libraries in ARM native code in the APK packages. Based on the DBT technique, the widely used Android emulator can run such ARM executables on x86-based platforms. Emulation performance is the key to the success of a cross-ISA DBT. Therefore, optimizing the translated code quality is important in the design of a cross-ISA DBT.

Single Instruction Multiple Data (SIMD) architectures are emerging as an essential extension to modern microprocessors, such as NEON for ARM, SSE/AVX for x86, AltiVec for PowerPC, and MSA for MIPS. The execution of a SIMD instruction simultaneously performs the same operation on multiple data, and is power and execution efficient. Recent trends in SIMD architecture development have also tended toward wider vector lengths. For instance, the Intel x86 architecture has gone through several generations: from 128-bit SIMD with SSE to 256-bit with AVX, to the most recent extension, AVX-512; ARM also plans to expand its ARMv8 architecture with scalable vector extension (SVE), where the vector length is up to 2,048 bits. In addition, more enhanced SIMD features are introduced in newer SIMD ISAs, e.g., fused multiply-and-add (FMA) and gather/scatter instructions. Many applications, such as multimedia processing and scientific computing, have enabled SIMD computations and achieved performance improvements of several orders of magnitude. We can expect that more applications utilizing SIMD ISA will be developed.

The importance of SIMD increases the need for supporting efficient SIMD translation in a DBT system. Several prior works have proposed solutions to support translation of SIMD instructions in cross-ISA DBTs. QEMU (Bellard 2005) and the Android emulator emulate a SIMD instruction by using a sequence of scalar instructions instead of leveraging host SIMD instructions. Other works (Fu et al. 2015; Li et al. 2006; Michel et al. 2011) focus on finding effective mappings of SIMD registers and instructions between different guest and host SIMD architectures. In these DBT systems, guest SIMD instructions are translated to equivalent host SIMD instructions that use *the same* number of lanes (e.g., ARM NEON instruction `vadd.i32` is translated to x86 SSE `paddl`; both are for four-integer vector addition).

Unfortunately, for legacy application binaries to run on host architectures with longer SIMD lanes, these works still translate guest SIMD instructions to host SIMD instructions for the same SIMD width. As a result, these approaches cannot benefit from the higher parallelism provided by the longer SIMD lanes, resulting in only a small fraction of the potential peak performance.

This article seeks to exploit full parallelism of longer host SIMD lanes to accelerate the execution of guest SIMD instructions. We propose a DBT system that enables *short-SIMD* binaries to exploit *long-SIMD* architectures by rewriting short-SIMD binary code. We aim to transform short-SIMD loops into equivalent long-SIMD loops optimized with long-SIMD instructions. By increasing the number of SIMD elements processed in each loop iteration, the execution time can be shortened. This work focuses on rewriting *loops* that consist of short-SIMD instructions; the loops before and after transformation are respectively denoted as short-SIMD and long-SIMD loops. We also improve the translated code quality by exploiting new SIMD features in our DBT.

Designing such a loop transformation involves several issues when applying a longer vector. First, how will the loop control be managed? This includes adjustment of long-SIMD loop iterations

and situations where the number of short-SIMD loop iterations is not divisible by the expanded factor. Second, there is a legality issue wherein a short-SIMD loop is not always rewritable to a long-SIMD loop due to a memory dependence violation. This issue is more challenging for DBTs than for static compilers because DBTs usually translate small granularity binary segments. Based on the limited information from input code segments, DBTs may not always be able to determine whether the transformation would violate dependence at time of translation. Third, SIMD architectures usually require data to be aligned to certain boundaries for efficient memory accesses. This causes the problem where memory accesses aligned in the original binary may become misaligned when translated to long-SIMD instructions. Therefore, how to enforce SIMD load/store alignment remains a challenging issue. The main contributions of this article are as follows:

- We present the dynamic binary translation technique to rewrite short-SIMD code to long-SIMD instructions. We propose a general approach that translates short-SIMD loops to machine-independent IR, conducts SIMD loop transformation at this IR level, and finally translates to host long-SIMD instructions.
- We identify the design challenges of memory dependence verification in a DBT. We present an approach of runtime memory checks and optimization to eliminate runtime checking overhead with compiler annotation.
- We identify the alignment issue in which aligned memory accesses can become misaligned when running in the long-SIMD loops. Two solutions, one for the problem caused by DBT internal translation conditions and one general approach using dynamic loop peeling, are presented to enforce SIMD load/store alignment.
- We implement the SIMD loop transformation in a cross-ISA DBT. Performance results of short-SIMD (ARM NEON) to long-SIMD (x86 AVX2, AVX-512) translation show that average speedups of 1.51 $\times$ and 2.48 $\times$ are achieved by doubling and quadrupling the SIMD lanes, respectively.

This article extends our previous work (Hong et al. 2016), which focuses on the method of SIMD loop transformation. The extensions include a new compiler annotation technique to eliminate runtime overhead and new experiments to compare with the alignment handling method that uses SIMD packing/unpacking operations. The rest of this article is organized as follows. Section 2 presents the approach of short-SIMD to long-SIMD loop transformation. Section 3 describes memory dependence verification and compiler annotation. Section 4 describes alignment problems and our solutions. Section 5 reports experimental results. Section 6 lists related work, and Section 7 concludes.

2 SHORT-SIMD TO LONG-SIMD LOOP TRANSFORMATION

2.1 The DBT Framework

The SIMD loop transformation is designed in HQEMU (Hong et al. 2012), a retargetable DBT that integrates QEMU and an LLVM JIT compiler (Lattner and Adve 2004). HQEMU runs a two-phase translation. In the first phase, the translator performs a fast translation, where the guest binary is translated to the host code with moderate optimization. When a hot execution path is detected by the translator, in the second phase the binary of the hot path is translated to LLVM IR (intermediate representation), which is aggressively optimized for higher performance.

In HQEMU, hot paths are detected by using the Next Executing Tail (NET) selection (Duesterwald and Bala 2000), a popular trace formation algorithm to detect cyclic execution paths (i.e., loops). NET is implemented by associating each guest basic block with one counter and a prediction buffer is maintained. The counter is incremented each time its associated translation

```

void
foo (float *x, float *y, float *z)
{
    for (int i = 0; i < 1028; i++)
        z[i] += x[i] * y[i];
}

```

(a) Source code

```

L1:
vld1.32 {d16-d17}, [r3]
vld1.32 {d18-d19}, [r1]!
vld1.32 {d20-d21}, [r2]!
vmla.f32 q8, q10, q9
vst1.32 {d16-d17}, [r3]!
cmp     r3, r0
bne     L1

```

(b) ARM32 NEON assembly

ShortSIMDLoop (SSL):

```

%ptr1 = phi [ %r1, %Entry ], [ %ptr1.next, %SSL ]
%ptr2 = phi [ %r2, %Entry ], [ %ptr2.next, %SSL ]
%ptr3 = phi [ %r3, %Entry ], [ %ptr3.next, %SSL ]
%vec1 = load <4 x float> *%ptr1    // x[i:i+4]
%ptr1.next = add %ptr1, 16
%vec2 = load <4 x float> *%ptr2    // y[i:i+4]
%ptr2.next = add %ptr2, 16
%vec3 = load <4 x float> *%ptr3    // z[i:i+4]
%vec4 = fmul <4 x float> %vec1, %vec2
%vec5 = fadd <4 x float> %vec3, %vec4
store %vec5, <4 x float> *%ptr3
%ptr3.next = add %ptr3, 16
%cond = icmp eq %ptr3.next, %end.4112
br %cond, label %Exit, label %SSL

```

(c) Short-SIMD loop IR

LongSIMDLoop (LSL):

```

%ptr1 = phi [ %r1, %Entry ], [ %ptr1.next, %LSL ]
%ptr2 = phi [ %r2, %Entry ], [ %ptr2.next, %LSL ]
%ptr3 = phi [ %r3, %Entry ], [ %ptr3.next, %LSL ]
%vec1 = load <8 x float> *%ptr1    // x[i:i+8]
%ptr1.next = add %ptr1, 32
%vec2 = load <8 x float> *%ptr2    // y[i:i+8]
%ptr2.next = add %ptr2, 32
%vec3 = load <8 x float> *%ptr3    // z[i:i+8]
%vec4 = fmul <8 x float> %vec1, %vec2
%vec5 = fadd <8 x float> %vec3, %vec4
store %vec5, <8 x float> *%ptr3
%ptr3.next = add %ptr3, 32
%cond = icmp eq %ptr3.next, %end.4096
br %cond, label %Exit, label %LSL

```

(d) Long-SIMD loop IR

Fig. 1. An example of loop transformation from ARM NEON to x86 AVX2. A vector type is represented as $\langle \#elements \times element\ type \rangle$.

block is executed. As the counter reaches a hot threshold, the guest blocks along with the following execution path are inserted into the prediction buffer. When the next guest block to insert is already in the prediction buffer, a cyclic execution path is found and a hot loop is recognized. The complete design of the NET trace formation in HQEMU is discussed in Hong et al. (2012).

Since loops are usually good vectorization candidates for static compilers and our DBT also selects loops for LLVM optimization, this work focuses on loops as the SIMD transformation candidates. We perform the SIMD transformation on LLVM IR. The goal is to convert IR of short-SIMD loops to equivalent long-SIMD loop representation. As a result, host long-SIMD instructions are emitted to exploit the advantages of the host SIMD architecture.

2.2 An Example of Short-to-Long-SIMD Loop Transformation

Figure 1 shows a transformation example from a short-SIMD loop (ARM32 NEON, vector size = 16B) to a long-SIMD loop (x86-64 AVX2, vector size = 32B). Figure 1(a) lists the loop source code. The source code is compiled by the static compiler (GCC) to a scalar loop, a NEON vector loop, and a runtime pointer checking block in the binary. The assembly code of the NEON vector loop is shown in Figure 1(b). In this example, base pointers of array x , y , and z are respectively mapped to registers $\%r1$, $\%r2$, and $\%r3$, and four consecutive single-precision floating-point data elements are executed at a time with NEON. Figure 1(c) shows the short-SIMD loop IR translated from the NEON loop binary by our DBT.

From the source code in Figure 1(a), we can observe that this loop can also be vectorized with AVX2. Since AVX2 can simultaneously operate on eight float data elements, we would like to transform the short-SIMD loop IR to execute eight elements at a time instead of four in the original

binary. The AVX2 vector length is twice the NEON vector. Thus, ideally we will double the number of vector elements of each IR vector type, modify the increasing value of memory pointers from 16 to 32, and adjust the loop trip count. The finalized long-SIMD loop IR is shown in Figure 1(d).

The example in Figure 1 shows that several issues have to be solved for the transformation. First, as the vector length is widened, the long-SIMD loop will compute more data elements in a single loop iteration. This implies that the number of loop iterations should be reduced. Moreover, there are situations in which the short-SIMD loop iterations cannot be completely consumed by the long-SIMD loop. For instance, the last index the long-SIMD loop can reach is 1,024 instead of 1,028 (i.e., 4,096 instead of 4,112 in byte units). The remaining iterations still need to be handled by the short-SIMD loop. This raises the issue of managing loop controls of both short-SIMD and long-SIMD loops.

Second, static compilers need to resolve the correctness issue regarding memory dependencies when performing vectorization. This issue also needs to be verified in our loop transformation. For example, if the addresses that are passed by argument z and x have a dependence distance of 16 bytes, it is a legal distance for NEON vectorization but not for AVX2 (vector size is 32 bytes). Thus, the translated AVX2 loop should not be executed and an additional check must be done to prevent its execution.

Third, the SIMD architecture usually requires that data be aligned to certain boundaries for efficient memory accesses, usually based on the vector size. That is, memory accesses with the short vector are restricted to a small value of the alignment boundary, but the long vector needs a larger value. As a result, memory accesses aligned in the original binary may become misaligned after the transformation. For example, assume the situation where array x is aligned at a 16-byte boundary but not at a 32-byte boundary. The accesses to x in the short-SIMD loop are always aligned, but those with a long-SIMD load (AVX2) are always misaligned because x is not aligned at the 32-byte boundary. Such a misalignment penalty could nullify the performance gain from the enhanced parallelism of the long-SIMD loop.

2.3 Approach of the SIMD Loop Transformation

The input of SIMD loop transformation is IR of a short-SIMD loop. We first scan the IR to see if SIMD memory access patterns are contiguous. This work only considers contiguous memory accesses and we leave the optimization of non-contiguous memory accesses for future work. After the IR is verified, the short-SIMD loop IR can be transformed to a long-SIMD loop, though it might not be executed due to violation of memory dependencies. In front of the short-SIMD loop, we add five additional blocks: RuntimeMemoryCheck, LongSIMDProlog, ShortSIMDLoopPeeling, LongSIMDLoopBody, and LongSIMDEpilog. Figure 2 shows the diagram of the transformed blocks.

In the following description, we use the terms *ShortVS* and *LongVS* to respectively represent the vector size of the short and long vectors. *Expand* represents the expanded vector factor, which is computed from

$$Expand = LongVS / ShortVS.$$

Before generating the long-SIMD loop, we must compute the start and end iteration of the long-SIMD loop. From analyzing the indexing variable (i.e., a PHI node) of the short-SIMD loop, we can retrieve three loop control values—*ShortStart*, *ShortEnd*, and *ShortStep*—which are, respectively, the start iteration, end iteration, and step value of the short-SIMD loop. The control values of the long-SIMD loop—*LongStart*, *LongEnd*, and *LongStep*—are computed with

$$\begin{aligned} LongStep &= ShortStep \times Expand, \\ LongStart &= ShortStart, \\ LongEnd &= ShortEnd - ((ShortEnd - ShortStart) \% LongStep), \end{aligned}$$

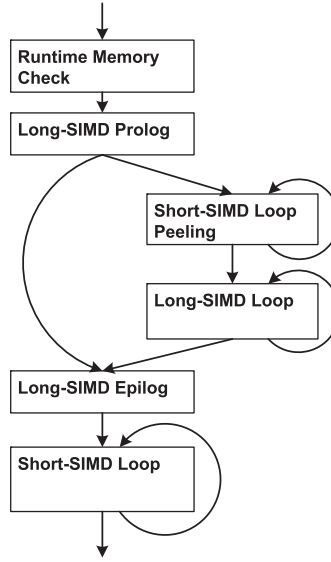


Fig. 2. Block organization and control flow of the transformed short-SIMD and long-SIMD loops.

and these calculations are inserted in LongSIMDProlog. Now, the long-SIMD loop body is populated by replicating IR of the short-SIMD loop with the following adjustments: the control of long-SIMD loop iterations is replaced with the computed three control values, and the number of vector elements is multiplied by *Expand* for each vector type.

Using the motivating example of the NEON to AVX2 transformation in Figure 1(c), the value *Expand* is 2 (32-byte/16-byte). The short-SIMD loop's three control values (%r3, %end.4112, 16) are derived from the indexing variable, %ptr3, by scalar evolution analysis. The long-SIMD loop is generated with the new three control values (%r3, %end.4096, 32) and the number of vector elements is updated to eight, as shown in Figure 1(d).

In block LongSIMDProlog, in addition to computing control values of the long-SIMD loop, we add a small test for the number of iterations, *LongEnd* – *LongStart*. We force the execution directly going to the short-SIMD loop instead of the long-SIMD loop if the trip count is too small. In block LongSIMDEpilog, we need to calculate the resuming iteration from which the short-SIMD loop will start. This resuming value can be either *ShortStart* if it comes directly from LongSIMDProlog or the end iteration, *LongEnd*, from the long-SIMD loop. The short-SIMD loop will resume from this value or just skip if the long-SIMD loop has finished all loop iterations. In the example of Figure 1(d), the long-SIMD loop can only run until the end value 4,096 is reached, and the short-SIMD loop will finish the last iteration.

The aforementioned indexing variable is also an induction variable. For inductions other than the indexing variable, e.g., %ptr1 and %ptr2, the rewriting scheme is similar to that of the indexing variable. SIMD reduction variables are handled similarly to the static compilers for scalar-to-vector transformation. The rewriting scheme is as follows: (1) A long vector variable is initialized for each long-SIMD reduction in LongSIMDProlog, and its initial value is set up by the initial value of the corresponding short-SIMD reduction. (2) The long-SIMD loop performs vector reduction operations on such long vector variables. (3) Each resultant long-SIMD reduction is split into partial short-SIMD reductions in LongSIMDEpilog. The final result is computed by performing short-SIMD reduction operations on all partial reductions. (4) The resuming reduction value of the short-SIMD loop is updated with the final result computed in (3).

3 MEMORY DEPENDENCE VERIFICATION

Vectorization entails changing the order of operations for the purpose of composing SIMD instructions. Vectorization is only legal if this change of order does not violate data dependencies. Since vectorizing memory operations can also change the data access order, static compilers need to verify memory dependence when performing vectorization, and so does our SIMD loop transformation. A number of dependence analysis techniques have been proposed in the literature (Banerjee 1976; Bulić and Guštin 2005; Kong et al. 1991; Wolfe and Tseng 1992; Zima and Chapman 1991). These techniques can be categorized into four schemes: (1) dependence hints with compiler directives, e.g., using type qualifier *restrict* in C99 or later to explicitly indicate no pointer aliasing, or instructing the Intel C compiler to ignore potential vector dependencies with *#pragma ivdep*; (2) simple numerical test, which attempts to prove the independence of memory references with low computational cost, e.g., GCD test; (3) pointer overlap test with loop bounds, which computes the memory region (i.e., the lowest and highest address) each memory operation can touch within the loop—dependence is determined by whether memory regions overlap or not; (4) distance test against the SIMD vector length (Bulić and Guštin 2005), which is for contiguous and unit-stride memory accesses—dependence is permitted for SIMD parallelism if the distance between memory references is *greater than or equal to* the SIMD vector length. These schemes have different guarantees on the independence of memory references. As two memory references are proved independent, schemes 1 to 3 guarantee that the memory references will never overlap across all loop iterations, while scheme 4 only ensures that the dependence does not occur within a limited number of iterations.

The legality of vectorization is often determined at compile time by the static compiler because the semantic information for dependence analysis, e.g., bounds of memory regions or distances, often can be resolved from the high-level programming language. Dynamic testing of memory references is only required for those undetermined cases. However, designing memory dependence analysis is more challenging in a DBT than in a static compiler, because the DBT (1) deals with binaries where high-level information may be lost and (2) cannot afford expensive analysis as in the static compilers due to the contention for runtime resources. In the following subsections, we first discuss the challenges of memory dependence verification in a DBT. We then present our approach of memory checks and the optimization to eliminate runtime testing overhead with compiler annotation.

3.1 Issues in Memory Dependence Verification

When a loop is compiled to the binary with dynamic testing code, some semantic information cannot be resolved statically by the compiler. It is very likely that the DBT cannot resolve the same information, and thereby runtime memory checking needs to be conducted for the transformed long-SIMD loop.

For a binary including no dynamic tests, though all the semantic information is known by the compiler, it is not always true that memory dependencies are still determinable when a SIMD loop is transformed. This is because, unlike static compilers, which have a complete view of the source programs, DBTs usually translate binary code segments of a small granularity, e.g., a basic block or a trace (single-entry, multiple-exit region). The dependence analysis is limited by how much information can be retrieved from the code segment. Figure 3 shows one example of such an issue. In Figure 3(a), the global arrays are directly accessed in the loop body. At the time of compilation, array addresses are resolved and there is no dependence between the array references according to the dependence analysis scheme (applying either scheme 3 or 4). This loop is vectorized and no dynamic testing is performed.

```

float x[1024], y[1024], z[1024];
void foo() {
    for (int i = 0; i < 1024; i++)
        z[i] += x[i] * y[i];
}

```

(a) Source code

<pre> L1: movaps 0x80496c0(%eax), %xmm0 add \$16, %eax mulps 0x804a6d0(%eax), %xmm0 addps 0x804b6f0(%eax), %xmm0 movaps %xmm0, 0x804b6f0(%eax) cmp \$4096, %eax jne L1 </pre>	<pre> ShortSIMDLoop (SSL): %eax = phi [%eax.init, %Entry], [%eax.next, %SSL] %ptr.y = add %eax, 0x80496c0 %eax.next = add %eax, 16 %ptr.x = add %eax, 0x804a6e0 %ptr.z = add %eax, 0x804b700 %vec1 = load <4 x float> * %ptr.x // x[i:i+4] %vec2 = load <4 x float> * %ptr.y // y[i:i+4] %vec3 = load <4 x float> * %ptr.z // z[i:i+4] %vec4 = fmul <4 x float> %vec1, %vec2 %vec5 = fadd <4 x float> %vec3, %vec4 store %vec5, <4 x float> * %ptr.z %cond = icmp eq %eax.next, 4096 br %cond, label %Exit, label %SSL </pre>
---	---

(b) x86-32 SSE assembly code and translated short-SIMD loop IR

<pre> L1: vld1.64 {d16-d17}, [r3 :64] vld1.64 {d18-d19}, [r1 :64]! vld1.64 {d20-d21}, [r2 :64]! vmla.f32 q8, q10, q9 vst1.64 {d16-d17}, [r3 :64]! cmp r3, r0 bne L1 </pre>	<pre> ShortSIMDLoop (SSL): %ptr1 = phi [%r1, %Entry], [%ptr1.next, %SSL] %ptr2 = phi [%r2, %Entry], [%ptr2.next, %SSL] %ptr3 = phi [%r3, %Entry], [%ptr3.next, %SSL] %vec1 = load <4 x float> * %ptr1 // x[i:i+4] %ptr1.next = add %ptr1, 16 %vec2 = load <4 x float> * %ptr2 // y[i:i+4] %ptr2.next = add %ptr2, 16 %vec3 = load <4 x float> * %ptr3 // z[i:i+4] %vec4 = fmul <4 x float> %vec1, %vec2 %vec5 = fadd <4 x float> %vec3, %vec4 store %vec5, <4 x float> * %ptr3 %ptr3.next = add %ptr3, 16 %cond = icmp eq %ptr3.next, %r0 br %cond, label %Exit, label %SSL </pre>
--	--

(c) ARM32 NEON assembly code and translated short-SIMD loop IR

Fig. 3. A loop example of global arrays compiled with x86-32 SSE and ARM32 NEON. Array base addresses are encoded in the x86 instructions and dependence verification can be done at translation time. The array addresses are not encoded in the ARM instructions due to the addressing mode of NEON memory instructions, and thus dependence is checked at runtime.

Figure 3(b) shows the compiled x86 SSE binary and the short-SIMD loop IR translated by our DBT. As the figure shows, the base addresses of global arrays are directly encoded in the x86 instructions within the SSE loop, and also translated in the short-SIMD loop IR. From this IR, the DBT computes access patterns of the memory pointers (i.e., $0x80496c0 + \%eax$, $0x804a6e0 + \%eax$, and $0x804b700 + \%eax$) and concludes that dependence distances between all memory references are always of fixed large values and never overlap. This SSE loop can be safely transformed to wider SIMD ISAs, such as x86 AVX2 and AVX-512, without inserting any runtime checking code.

Figure 3(c) shows the ARM32 NEON binary and translated IR from the same source code. In contrast to the SSE binary, array base addresses are not encoded in the NEON loop. This is because such address encoding is not supported by the addressing mode of NEON load/store instructions. Instead, array base addresses are loaded into the general-purpose registers ($\%r1$, $\%r2$, and $\%r3$) before entering the loop body. Thus, array address information is lost in the short-SIMD loop IR.

Furthermore, compared with the example of passing array references by function arguments in Figure 1, the NEON loop binaries in both examples are almost the same (the only difference is that the compiler knows the alignment of global arrays and selects wider-aligned memory instructions such as `vld1.64` in this example); the same short-SIMD loop IR is also translated. Based on this IR, the DBT is unable to distinguish in which way the array addresses are passed. One possible solution is to search for array addresses from the assignment instructions at predecessor blocks. However, the addresses found cannot be used for dependence analysis because we cannot ensure if any other execution path could jump to this short-SIMD loop, which may conflict with the analyzed result. As a consequence, runtime memory checking is required for this example, and thus incurs additional overhead.

Another issue arises from the dependence analysis scheme conducted by static compilers. When DBTs perform SIMD loop transformation, this implies that the short-SIMD loop has passed dependence tests either from static analysis or from dynamic testing. Two conditions are for the SIMD transformation: (1) for loops derived from distance tests (i.e., scheme 4), additional distance tests must be done against the long-SIMD vector size because the legality for long SIMD is not evaluated by the static compiler, and (2) further testing can be skipped and the transformed long-SIMD loops are still correct when memory references never overlap (i.e., scheme 1 to 3)—that is, the transformation is independent of the SIMD vector size. However, the DBT usually cannot determine which testing scheme is applied by static compilers at the binary level. Therefore, to be conservative, we must assume all loops could violate dependencies against the long-SIMD vector size and all short-SIMD loops must be verified.

3.2 Approach of Memory Checks

The memory dependence verification is conducted with distance tests against the long-SIMD vector size. Since our transformation focuses on SIMD loops of contiguous memory accesses, this distance testing is sufficient to cover all testing schemes by static compilers and verify the correctness of transformed long-SIMD loops.

Algorithm 1 shows the steps of memory dependence checks applied at the time of transformation. At first, SIMD memory operations that are collected from short-SIMD loop IR are converted into a list of memory reference pairs, where each pair contains two references $\langle p, q \rangle$ and the instruction order of p is less than q (line 2). Then, the distance, $q - p$, is calculated for each read-write and write-write pair. If the distance of a memory reference pair is not computable (i.e., calculated result is not a constant value; e.g., the distance of `<load x, store z>` is `%r3 - %r1` in the NEON loop example in Figure 3(c)), this reference pair is recorded and whether p and q have dependence will be checked at runtime (lines 7–9). For a computable distance, it is compared with the long-SIMD vector size, `LongVS`. There are three conditions: (1) a negative or zero distance is not a plausible dependency; (2) a distance greater than or equal to the long-SIMD vector size is permitted for the long SIMD; and (3) if a distance falls into the range of the long-SIMD vector size, a dependency is detected—this loop cannot be transformed to long SIMD and the loop transformation aborts (lines 11–14).

For memory reference pairs of undetermined distances, they are verified at runtime based on the same distance testing in the algorithm (lines 11–14). The verification code is inserted in block `RuntimeMemoryCheck`. Any dependence detection will force the execution to jump directly to the short-SIMD loop.

3.3 Compiler Annotation

From the example in Figure 3, we observe that whether a runtime dependence check can be optimized or not depends on how much information DBTs can retrieve from the binary code segments.

ALGORITHM 1: Memory Dependence Check

Input: List of short-SIMD memory references M , short-SIMD vector size w , long/short-SIMD expand factor s .

Output: Return true if dependence is detected; otherwise, return false.

```

1 Function MemoryCheck( $M$ )
2    $MPairs \leftarrow$  a list of memory reference pairs  $\langle p, q \rangle$  computed from  $M$ , where instruction order
    $p < q$ 
3   foreach Memory reference pair  $\langle p, q \rangle \in MPairs$  do
4     if  $p$  and  $q$  are both memory load operations then
5       continue
6     end
7     if distance,  $q - p$ , is not computable then
8       call addRuntimeCheckPair( $p, q$ ) and continue
9     end
10
11      $Distance \leftarrow q - p$ 
12     if  $Distance > 0$  and  $Distance < s w$  then
13       return true /* Dependence is detected. */
14     end
15   end
16   return false
17 Function addRuntimeCheckPair( $p, q$ ): add a memory reference pair  $\langle p, q \rangle$  for runtime check.

```

Although the dependence is determined by static compilers at the time of compilation, static information such as array base addresses could be lost due to the capability of the ISAs (e.g., addressing mode) or compiler optimizations (e.g., moving the instructions that contain address information out of the SIMD loop body via code motion). DBTs usually have difficulty to reconstruct the semantic information because (1) it could be lost at the binary level, (2) it is not included in the loop code segment to be transformed, or (3) the overhead of information reconstruction is unacceptable for a DBT.

In contrast, static compilers that have a complete view of the source programs can extensively analyze loops and array structures from the higher language level. In addition, programmers can provide hints to assist compilers with analyzing dependencies and generating high-performance vector code, e.g., using Intel-specific pragmas *ivdep*, *simd*, and *vector*. This motivates us to propose the approach in which static compilers provide critical information via compiler annotation to assist DBTs in optimizing runtime memory checking. Although this approach requires assistance from static compilers, which can access the program source and recompile the program, there are situations where the widest SIMD lanes of the guest are narrower than the host SIMD ISAs, e.g., emulating ARM NEON binaries on x86 AVX hosts. In such a scenario, recompiling the programs for the guest ISA is infeasible to achieve the maximum parallelism of the host SIMD architecture. It still needs SIMD loop transformation to exploit host SIMD parallelism via DBT.

During SIMD loop transformation, the DBT checks dependence distances between memory references—the same distances that are also computed by the static compiler but discarded after the loop is vectorized. It is helpful if the compiler can annotate such calculated results when all dependence distances are compile-time computable. The distance of one memory reference pair is annotated as

$$DistanceAnnotation := \langle MemoryRefP, MemoryRefQ, DependenceDistance \rangle,$$

where $MemoryRef\{P, Q\}$ refers to the locations of the memory instructions in the binary and $DependenceDistance$ is a positive value. For a negative or zero distance, $DependenceDistance$ is set to *infinity*, meaning no dependence. Take the NEON loop in Figure 3(c) as an example. Assume global array x , y , and z are located at address $0x496c0$, $0x4a6e0$, and $0x4b700$. The distance of the memory reference pair $\langle load\ x, store\ z \rangle$ (the second and fifth instruction) is a computable value, $0x2040$. This reference pair is annotated as $\langle L1+0x4, L1+0x10, 0x2040 \rangle$.

Considering the hints that programmers annotate to help dependence analysis, we also would like compilers to pass such information for DBTs. For instance, in the address passing example of Figure 1, programmers can declare pointers with the *restrict* keyword or put *#pragma ivdep* before the loop to inform the compiler that memory references are not aliased. For such a case, the compiler can annotate $\langle L1+0x4, L1+0x10, infinity \rangle$ to indicate there is no dependence between the pair $\langle load\ x, store\ z \rangle$.

However, annotating distance information for all memory reference pairs is not acceptable because the number of distance information will be excessive when there are a large number of memory references. To overcome this problem, recall that we conduct memory dependence verification with the distance testing scheme based on the rule of whether the distance is greater than or equal to the long-SIMD vector size. This implies that comparing the *minimum distance* with the long-SIMD vector size is sufficient for the dependence testing for all distance annotations in the short-SIMD loop. For instance, in the NEON loop example mentioned earlier, the minimum distance is $0x1020\ (y - x)$, which is greater than the long-SIMD vector size (32B in AVX2); the other pairs must have the same comparison results. Hence, a short-SIMD loop is annotated with only one value:

$$DistanceAnnotation := \langle \min_i DependenceDistance(i) \rangle.$$

By comparing the annotated dependence distance with the long-SIMD vector size, the DBT can decide to abort SIMD loop transformation in the first place if the dependence distance is too short. When the distance is legal, block RuntimeMemoryCheck can be removed and runtime checking overhead is eliminated as a result.

4 ALIGNMENT PROBLEM

Recall that during SIMD loop transformation, short-SIMD load/store instructions are rewritten to long-SIMD instructions by expanding the access size. A question is raised here: which long-SIMD memory instruction is translated with regard to the alignment constraints? That is, should the DBT select aligned or unaligned SIMD memory instructions? Most SIMD architectures are designed with alignment constraints, usually based on the vector size, e.g., 16-byte boundary for NEON/SSE and 32-byte for AVX2. A short-SIMD load/store instruction of the aligned version could not be mapped to an aligned long-SIMD version because the long SIMD has a stronger alignment constraint. Instead, it is translated to the unaligned version; e.g., NEON `vld1.64` instruction is translated to the unaligned memory instruction `vmovups` of AVX2. For short-SIMD load/store instructions of the unaligned version, they are also translated to unaligned long-SIMD ones.

Although we use unaligned long-SIMD memory instructions, the long-SIMD loads/stores can still incur a significant performance penalty if the accessed data are not properly aligned with the alignment constraint of the long-SIMD ISA. This is because the accessed data of short-SIMD binaries are aligned according to the short-SIMD alignment constraint. When running on a long-SIMD machine, these data are very likely to be placed at memory locations with original alignment constraints and may not satisfy the alignment constraints of long-SIMD ISAs. Hence, data accesses with long-SIMD loads/stores become misaligned.

This misalignment situation can be relieved if data of the source programs are placed at a larger alignment boundary on the long-SIMD machines. As the data's alignment can satisfy the

```

#define ALIGN32 __attribute__((aligned(32)))

float x[1024] ALIGN32;
float y[1024] ALIGN32;
float z[1024] ALIGN32;
void foo() {
    for (int j = 0; j < 1024; j++) {
        for (int i = 0; i < 1024; i++)
            z[i] += x[i] * y[i];
    }
}

```

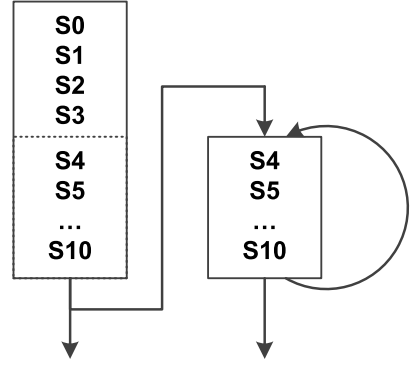
(a) Source code

```

S0: ldr      r2, [pc, #52]
S1: movw    r3, #51008
S2: movt    r3, #6
S3: add     r1, r2, #4096
S4: vld1.64 {d16-d17}, [r3 :64]
S5: vld1.64 {d18-d19}, [r1 :64]!
S6: vld1.64 {d20-d21}, [r2 :64]!
S7: vmla.f32 q8, q10, q9
S8: vst1.64 {d16-d17}, [r3 :64]!
S9: cmp     r3, r0
S10: bne    S4
S11: subs   ip, ip, #1
S12: bne    S0

```

(b) ARM32 NEON assembly code



(c) Code fragments decoded by DBT

Fig. 4. A nested loop example with NEON-to-AVX2 transformation. The problem of code duplication causes the long-SIMD loop to execute the inner loop starting from the second iteration, and thus long-SIMD loads/stores become misaligned.

long-SIMD platform's alignment requirement, transformed long-SIMD loads/stores will not cause any misaligned accesses. For example, programmers can force data to align at 64-byte boundaries. The transformed binaries can run on AVX2 and AVX-512 machines (the alignment boundaries are 32-byte and 64-byte, respectively) without incurring misaligned accesses. To verify this scenario, we perform an experiment on a nested loop with NEON-to-AVX2 translation. The source code is listed in Figure 4(a). In the source code, global arrays are aligned at 32-byte boundaries, which match the alignment constraint of AVX2. The transformed long-SIMD loop should ideally have no misaligned memory accesses to the global arrays. Surprisingly, the long-SIMD loads/stores are all misaligned (profile data are listed in Section 5.4). The details of this problem are described in the next subsection.

4.1 DBT Termination Condition

When an untranslated binary code is encountered, a DBT starts to decode and translate a fragment of binary code. In the decode phase, it has to decide when the binary decoding should be terminated. A DBT usually stops decoding binary code when a control transfer instruction is encountered, e.g., a branch, call, or return instruction. Such a termination condition based on the control transfer instruction is widely applied in many DBTs (Bellard 2005; Bruening et al. 2003; Scott and Davidson 2001). Figure 4(b) shows the ARM32 NEON assembly code of the nested loop in Figure 4(a). As Figure 4(b) shows, the inner loop is composed of the instructions from S4 to S10, and the outer loop is from S0 to S12. The DBT first decodes the outer loop starting from S0 until

the first control transfer instruction, S10, is reached—thus, the first code fragment, S0 to S10, is translated. Then, the instruction S10 branches to S4 (i.e., head of the inner loop) and the second code fragment, S4 to S10, is decoded because the code starting from S4 is not yet translated. The second code fragment forms a loop since S4 is a branch target of instruction S10, but the first code fragment is not because it jumps to the middle of the fragment. These two code fragments and their control flows are illustrated in Figure 4(c). Because the second code fragment is a loop, it is further optimized with our SIMD loop transformation.

As we can see from Figure 4(c), both code fragments consist of binary code of the inner loop (S4 to S10). When execution starts from the outer loop, one iteration of the inner loop is executed by the first code fragment. As a result, the second code fragment, which is transformed to a long-SIMD loop, will execute the inner loop starting from the second iteration. Hence, the array addresses accessed by long-SIMD loads/stores are shifted by one short-SIMD vector size. Memory accesses become misaligned even though the global arrays are aligned at long-SIMD alignment boundaries.

Such misalignment is caused by *code duplication* of the inner loop, and it can be overcome by preventing code duplication. The solution is to let a DBT stop binary decoding if it tries to decode an inner loop in the middle of a code fragment, e.g., decoding stops at S3 with the example code. At the binary level, however, it is not easy for a DBT to determine whether the next instruction to decode is at a loop head or not. Even though the loop head information can be retrieved (e.g., with compiler annotation), this solution works well only when the data layout of short-SIMD binaries satisfies the alignment constraints of long-SIMD ISAs. It cannot solve cases where data are aligned at a smaller alignment boundary than the minimum constraint of long-SIMD ISAs. Hence, a general approach is called for.

4.2 Dynamic Loop Peeling

A general solution to enforce the alignment of long-SIMD loads/stores is to dynamically peel short-SIMD loops until memory references are aligned with long-SIMD alignment constraints. Therefore, we create a block ShortSIMDLoopPeeling in front of the long-SIMD loop body. The ShortSIMDLoopPeeling block is populated by replicating IR instructions from the short-SIMD loop. The next step is to decide the number of iterations for short-SIMD loop peeling. The peeling count is calculated with a voting algorithm as follows:

All pointers of SIMD memory references are collected from the short-SIMD loop. For each memory pointer, the distance between the referenced address and the next long-SIMD alignment boundary is computed; the peeling count of this pointer is the value of the distance divided by the short-SIMD vector size, shown in the formula

$$PeelCount = (LongVS - P \% LongVS) / ShortVS,$$

where P is the referenced address. Then, peeling counts of all memory pointers are conducted by a voting process to find the majority, which is the number of short-SIMD loop iterations to be executed in ShortSIMDLoopPeeling. With this voting scheme computed in block RuntimeMemoryCheck, we can ensure that as many long-SIMD loads/stores as possible are aligned. The algorithm is listed in Algorithm 2.

5 PERFORMANCE EVALUATION

5.1 Experimental Setup

We evaluate the performance of SIMD loop transformation from one guest ISA, ARM32 NEON, to two long-SIMD host ISAs, x86-64 AVX2 and x86-64 AVX-512. In the following sections, we use AVX-128 and AVX-256 to respectively represent AVX of 128-bit and 256-bit mode. The experiments are performed on three hardware platforms:

Table 1. Description of Benchmark Suites

Name	Description	#Check	Name	Description	#Check
Livermore Loops Benchmark Suite, C Version (LL)			OpenCV, Open Source Computer Vision Library (CV)		
hydro	Hydrodynamics fragment	3	laplacian	Calculate Laplacian of an image	13
inner	Inner product	2	sobel	Conduct Sobel edge detection	4
ADI	Alternating direction implicit	96	gaussian	Blur an image using the Gaussian filter	10
int_pred	Integration predictors	10			
diff_pred	Difference predictors	136			
first_diff	First difference	2			
PIC_1D	1-D particle in cell	24			
Test Suite for Vectorizing Compilers (TSVC)					
s231, s235, s3251, s2275, vbor	Test cases for loop restructuring, loop expansion, and loop controls	2~13	SM2-LU	Dense LU matrix factorization	3
			NPB-MG	Solve Poisson equation in 3D	31
			Linpack	Perform numerical linear algebra	9

ALGORITHM 2: Dynamic Loop Peeling

Input: List of short-SIMD memory references M , short-SIMD vector size w , long/short SIMD expand factor s .

Output: The number of iterations to peel k .

```

1  $K \leftarrow$  array of size  $s$  with zero initial values
2 foreach Memory reference  $p \in M$  do
3    $k \leftarrow (s \cdot w - (p \bmod (s \cdot w))) / w$ 
4    $K[k] \leftarrow K[k] + 1$ 
5 end
6 return  $\arg \max_k K[k]$ 
```

- **AVX-256:** one 6-core Intel Haswell Core i7-5930k 3.50GHz processor with 32KB L1D cache and 15MB last-level cache (L3). The host has 16GB main memory and runs 64-bit Gentoo Linux with kernel 3.6.15.
- **AVX-256:** one 4-core Intel Skylake Core i7-6700 3.40GHz processor with 32KB L1D cache and 8MB last-level cache (L3). The host has 4GB main memory and runs 64-bit CentOS Linux with kernel 4.6.0.
- **AVX-512:** one 64-core Intel Xeon Phi 7210 1.30GHz processor (Knights Landing) with 32KB L1D cache and 1MB last-level cache (L2). The host has 12GB main memory and runs 64-bit CentOS Linux with kernel 3.10.0.

Eighteen benchmarks are selected from various benchmark suites for the evaluation: Livermore loops (LL), the TSVC benchmark suite, NAS parallel benchmarks (NPB), SciMark 2 (SM2), Linpack, and OpenCV (CV). A brief description of the benchmarks is listed in Table 1. Since ARM32 NEON does not support double-precision floating-point SIMD operations, the benchmarks are configured with single precision for floating-point data type. All benchmarks are compiled using ARM GCC 4.8.4 with flags “-marm -march=armv7-a -mtune=cortex-a8 -static -O3 -fno-unroll-loops -mfpu=neon -ffast-math -ftree-vectorize.” LLVM 3.8 is used in our DBT framework and a default optimization level (-O2) is set for the LLVM JIT compilation. We take the median of 10 runs as

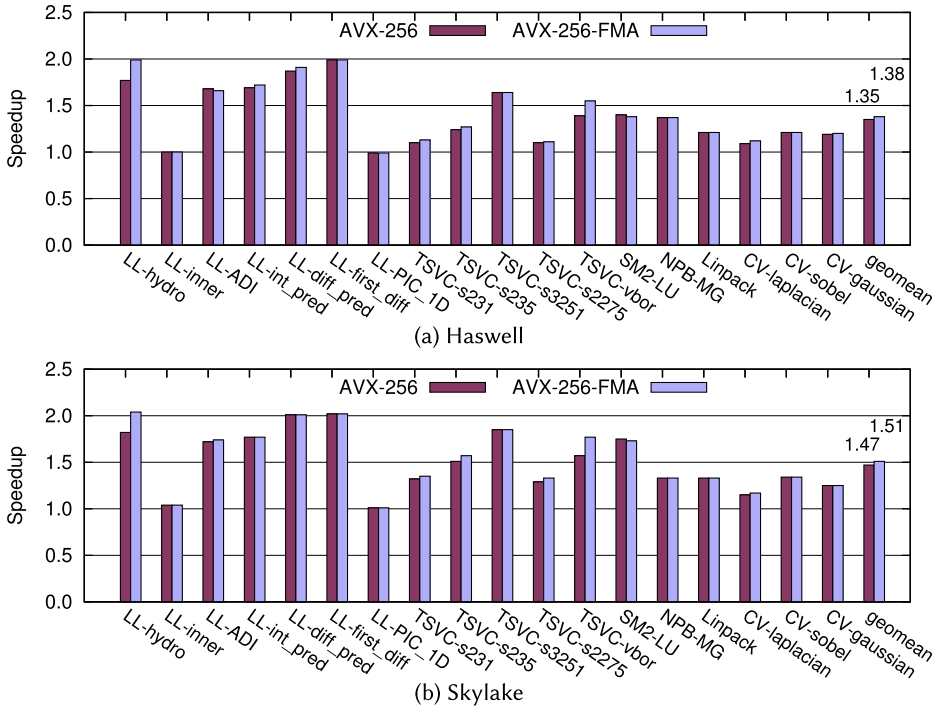


Fig. 5. Performance of NEON-to-AVX-256 loop transformation.

the final performance result. For comparison, the binary translation without SIMD loop transformation is used as the performance baseline, where ARM32 NEON instructions are translated to x86-64 AVX-128 instructions on the host machines. Dynamic loop peeling is enabled and compiler annotation is disabled for all experiments unless changes of the setting are mentioned.

5.2 Performance Results with AVX-256

Figure 5 shows the speedup of the NEON-to-AVX-256 loop transformation. With FMA disabled in LLVM JIT, the SIMD loop transformation results in average speedups of $1.35\times$ and $1.47\times$ over the baseline performance on Haswell and Skylake, respectively. Many of the benchmarks exhibit an improvement of over 50%, while two benchmarks, LL-inner and LL-PIC_1D, show the least performance gain.

The improvement that can be accomplished from long SIMD is affected by the following factors: (1) according to Amdahl's law, the speedup is limited by the ratio of non-SIMD code, and (2) composition of instructions in the SIMD loop, where the speedup is affected by the throughput of SIMD instructions. Table 2 lists the long-SIMD instruction throughput over AVX-128 on the experimental platforms. The throughput is measured by processing a fixed number of data elements with different SIMD lanes (we set the data elements to fit in the L2 cache for SIMD load/store measurement because L2 is the furthest cache level in common for the three architectures). As Table 2 shows, ideal throughput is achieved for most operations when increasing the number of SIMD lanes, e.g., common arithmetic (add, sub, mul), logical (and, or, xor), FMA, and load. The store performance is very different in each architecture due to different processing capabilities of the processor backend.

Table 2. SIMD Throughput over AVX-128

	Haswell	Skylake	Knights Landing	
	AVX256	AVX256	AVX256	AVX512
arith/logical/FMA	2×	2×	2×	4×
div	1×	1.2×	2×	4×
load	2×	2×	2×	4×
store	1×	1.5×	2×	2.3×

Benchmark LL-inner computes inner products of two arrays. The static compiler splits this computation into two loops: the first loop multiplies corresponding elements of two input arrays and stores results in an output array; the second loop iterates over each element of the output array and computes the final sum. Of the two loops, only the first loop is vectorized by the compiler and the second loop has no SIMD operations. We profile the execution of LL-inner without applying SIMD loop transformation. Profile data report that the first loop contributes to only about 15% of the total execution time, which is the maximum performance gain our optimization can achieve. In addition, the first loop is composed of only one SIMD multiplication and three SIMD loads/stores. According to Table 2, the throughput of SIMD store operations is identical on Haswell regardless of whether the SIMD store instructions are translated with AVX-128 or AVX-256. For SIMD computation instructions, rewriting them improves the performance with longer SIMD lanes; however, the amount of SIMD computations in the loop is not sufficient. As a result, the performance is bounded by SIMD stores due to high memory stall cycles. There is very little room for improvement on this loop, and thus no performance improvement is achieved after transformation. Similar behavior is also observed with benchmark LL-PIC_1D. In contrast, benchmarks such as LL-first_diff, LL-hydro, and TSVC-s3251 achieve significant performance improvement because their short-SIMD loops have a high ratio of SIMD computations.

Comparing the results of Skylake and Haswell, the SIMD loop transformation achieves better performance gains on Skylake for several benchmarks, e.g., LL-diff_pred and TSVC benchmarks. This is because these benchmarks have relatively higher ratios of SIMD stores than the other benchmarks. Due to the higher AVX-256 store throughput on Skylake than on Haswell (Table 2), a better improvement is achieved. Such a situation is also observed in LL-inner, which is slightly improved by 4%. On average, a speedup of 1.47× is achieved for all benchmarks.

To evaluate the impact of host machines' multiply-and-add instruction, we compare the performance of enabling and disabling FMA code emission in the LLVM JIT compiler. Note that there is no special FMA IR instruction supported in LLVM IR. Therefore, although ARM32 NEON ISA supports the FMA feature, a multiply-and-add instruction is translated into two LLVM IR instructions—a multiply instruction followed by an add instruction. The LLVM instruction selection pass can find such patterns and map them to one host FMA instruction. Figure 5 also shows the transformation results with FMA enabled. Performance improvement can be expected if the short-SIMD loop consists of multiply-and-add operations. Benchmark LL-hydro and TSVC-vbor have the most significant performance gain with FMA because they are the ones with the greatest number of multiply-and-add operations, for an improvement of 13%. The overall benchmark performance is further improved to 1.38× and 1.51× on average with benefits from host FMA instructions on Haswell and Skylake, respectively.

The results in this section show the effectiveness of our DBT system: not only can we exploit the improved parallelism from long-SIMD architectures with our SIMD loop transformation, but also we allow the applications to use new SIMD features with the DBT technique.

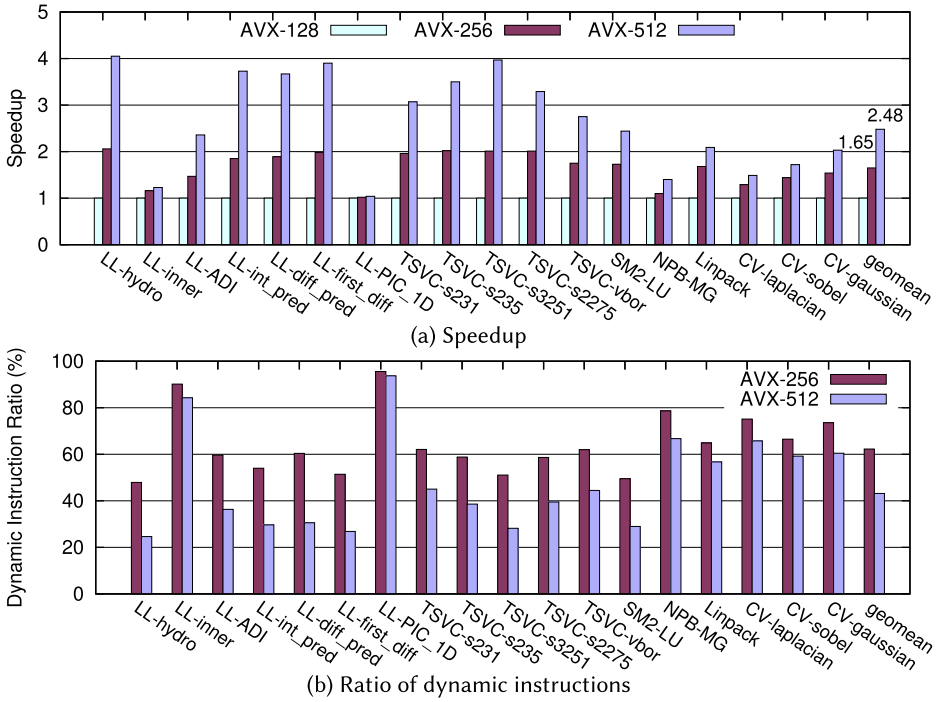


Fig. 6. Performance of NEON-to-AVX-256/AVX-512 loop transformation on Knights Landing.

5.3 Performance Results with AVX-512

Figure 6 shows the performance of NEON-to-AVX-512 loop transformation and the dynamic instruction counts measured with hardware performance counters on Intel Xeon Phi. The transformation of NEON to AVX-256 is also evaluated for comparison. As Figure 6(a) shows, average speedups of 1.65 $\times$ and 2.48 $\times$ are achieved with AVX-256 and AVX-512. For AVX-256 results, since the throughput of AVX-256 computation and load/store operations can reach the theoretical performance on Knights Landing (Table 2), several Livermore and TSVC benchmarks achieve the maximum speedup of 2 $\times$. The AVX-512 results outperform those of AVX-256 for all benchmarks but do not achieve the theoretical improvement because of Amdahl's law and the limitations from AVX-512 store performance. Slight improvement is also observed with LL-inner in this experiment.

Figure 6(b) shows the dynamic instruction ratios normalized to the baseline performance (i.e., instruction count of NEON-to-AVX-128 translation). Lower results are better. The results show that fewer instructions are executed when a longer SIMD is used. On average, AVX-256 and AVX-512 respectively reduce the numbers of dynamic instructions executed to 62% and 43% of that without loop transformation. The dynamic instruction ratios of LL-inner and LL-PIC_1D are only slightly reduced because their SIMD loops only contribute to a small proportion of the total instruction count. This also explains the performance results in Figure 5 and Figure 6 where no noticeable performance improvement is observed for either benchmark.

5.4 Comparison of Alignment Adjustment Approaches

In this section, we compare the performance of SIMD loop transformation with three configurations: (1) no alignment adjustment, (2) optimized DBT termination condition, and (3) dynamic

Table 3. Misalignment Ratio of (1) No Adjustment, (2) Optimized DBT Termination Condition, and (3) Dynamic Loop Peeling, with NEON-to-AVX-256 Loop Transformation

	Aligned with AVX-256 Constraint (32-Byte Boundary)			Aligned with NEON Constraint (16-Byte Boundary)		
Benchmark	NoAdjust	Term	Peeling	NoAdjust	Term	Peeling
LL-hydro	100%	50%	50%	75%	75%	75%
LL-inner	100%	0%	0%	67%	33%	33%
LL-ADI	100%	86%	86%	86%	95%	86%
LL-int_pred	100%	0%	0%	0%	100%	0%
LL-diff_pred	100%	0%	0%	0%	100%	0%
LL-first_diff	100%	33%	33%	67%	67%	67%
LL-PIC_1D	100%	0%	0%	0%	100%	0%
TSVC-s231	100%	0%	0%	0%	100%	0%
TSVC-s235	100%	0%	0%	75%	25%	25%
TSVC-s3251	100%	14%	14%	43%	71%	43%
TSVC-s2275	100%	0%	0%	75%	25%	25%
TSVC-vbor	100%	0%	0%	43%	57%	43%
SM2-LU	88%	88%	79%	88%	88%	75%
NPB-MG	64%	36%	20%	58%	36%	20%
Linpack	87%	87%	75%	87%	87%	75%

loop peeling. We first conduct the experiment with NEON-to-AVX-256 loop transformation. Because the alignment constraints of the guest and host SIMD ISA are different, we manually align source arrays at the addresses based on the host alignment constraint (i.e., AVX-256 with 32-byte boundaries) or on the guest alignment constraint (i.e., NEON with 16-byte boundaries but not 32-byte boundaries), and compare their performance. We profile the benchmarks to see if the transformed AVX-256 loads/stores are properly aligned at 32-byte boundaries. The misalignment ratio is measured as

$$Ratio = \frac{\text{Number of misaligned SIMD accesses}}{\text{Number of total SIMD memory accesses}}.$$

Table 3 lists the misalignment ratio. Note that benchmarks from OpenCV are not included in this experiment because we are unable to change the alignment of OpenCV image data. When source arrays are aligned based on the host AVX-256 constraint, the SIMD loads/stores in long-SIMD loops ought to be aligned. However, profile data show a 100% misalignment ratio for all Livermore loops and TSVC benchmarks without applying any alignment adjustment approach. These surprising results are caused by the DBT code duplication problem, and all accessed addresses become misaligned by one NEON vector size. By optimizing the DBT termination condition, most misaligned memory accesses are eliminated. The dynamic loop peeling approach also produces the same good results as the DBT termination condition optimization. Both approaches can achieve the smallest misalignment ratio. In Table 3, there are still misaligned accesses, e.g., an 86% misalignment ratio with benchmark LL-ADI. The reason is that those SIMD load/store patterns originally misaligned in the guest binary remain misaligned after the SIMD loop transformation, e.g., a SIMD load that always accesses at 4-byte aligned addresses. They cannot be eliminated by our alignment adjustment approaches. Figure 7 shows the performance speedup of two alignment adjustment approaches over that without adjustment. Benchmark LL-diff_pred achieves

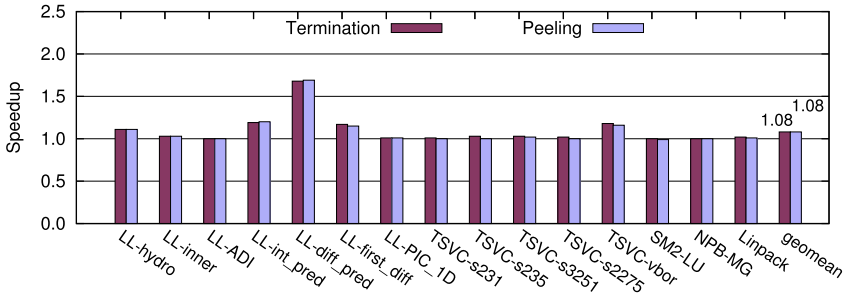


Fig. 7. Performance of alignment optimization approaches with NEON-to-AVX-256 loop transformation.

Table 4. Misalignment Ratio with NEON-to-AVX-512 Loop Transformation

Benchmark	NoAdjust	Term	Peeling	Benchmark	NoAdjust	Term	Peeling
LL-hydro	75%	75%	50%	TSVC-s3251	86%	86%	57%
LL-inner	67%	33%	33%	TSVC-s2275	100%	100%	75%
LL-ADI	92%	92%	87%	TSVC-vbor	86%	86%	57%
LL-int_pred	100%	100%	0%	SM2-LU	94%	94%	87%
LL-diff_pred	100%	100%	0%	NPB-MG	83%	76%	42%
LL-first_diff	67%	67%	67%	Linpack	94%	94%	80%
LL-PIC_1D	100%	100%	0%	CV-laplacian	96%	39%	31%
TSVC-s231	100%	100%	0%	CV-sobel	95%	31%	30%
TSVC-s235	100%	100%	75%	CV-gaussian	98%	38%	38%

a 70% performance improvement with alignment adjustment, and an average 8% improvement is achieved for all benchmarks.

When source arrays are aligned based on the guest NEON constraint, one expects that the transformed long-SIMD loads/stores will be misaligned. However, the code duplication caused by DBT coincidentally causes misaligned accesses to become aligned. On the contrary, applying the optimized DBT termination condition reverses this situation. We can see from Table 3 that the misalignment ratio of the optimized DBT termination condition is higher than that without adjustment for several benchmarks. With dynamic loop peeling, the DBT always adjusts long-SIMD loops to start accessing arrays from aligned addresses. Therefore, the dynamic loop peeling approach can achieve the minimum misalignment ratio regardless of whether the source arrays are aligned with the guest or host SIMD alignment constraint.

Table 4 further demonstrates the benefit of dynamic loop peeling. In this experiment, NEON-to-AVX-512 loop transformation is conducted. We do not force the alignment of source arrays, but rather allow the guest compiler to determine the placement of source arrays. Misalignment ratios of three configurations are listed in Table 4. As the host SIMD vector size increases to 4× that of the guest, the optimal peeling problem becomes more complex than a simple “peeling or not” decision as in the case of NEON-to-AVX-256 transformation. Therefore, changing the DBT terminating condition is not as helpful since the available peeling distance becomes $k = \{0, 1, 2, 3\}$ in AVX-512. It has only a 0.25 probability instead of 0.5 to “guess” the optimal one, while the dynamic peeling algorithm always selects the optimal $\hat{k} \in k$. The profile data in Table 4 show that dynamic loop peeling can achieve the minimal misalignment ratio. The performance results in Figure 8 also show a 43% improvement for LL-diff_pred with dynamic loop peeling.

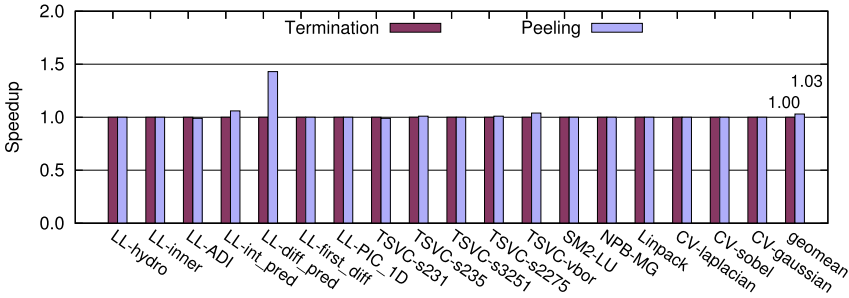


Fig. 8. Performance of alignment optimization approaches with NEON-to-AVX-512 loop transformation.

5.5 Comparison of Alignment Handling with SIMD Packing/Unpacking

In the previous section, we have demonstrated that the dynamic loop peeling approach can effectively minimize misaligned accesses. Unlike our approach, Hallou et al. (2015) proposed a different alignment handling scheme, which is based on SIMD packing/unpacking operations. The rewriting rules of the short-SIMD memory instruction in their system are as follows: (1) if the data address is provided in the code section as a static value and the data is also aligned at the boundary of the long-SIMD alignment constraint, an aligned host long-SIMD memory instruction is translated; (2) otherwise, the memory access is decomposed into multiple host short-SIMD load/store operations, and short-SIMD data are packed/unpacked into/from the long-SIMD register.

As the data addresses are resolved and aligned properly with the host alignment constraint, their approach can generate the same long-SIMD loop body as ours but uses smaller code cache size because an additional loop peeling routine is not required. However, there are often cases where the alignment of guest SIMD memory operations cannot be resolved at translation time, and thus multiple short-SIMD memory accesses plus packing/unpacking operations must be used. First, address information can be lost in the code section when the binary is stripped. Second, even though the address information is not stripped, the alignment cannot be determined when the address is passed by a function argument, e.g., the motivating example in Figure 1. Third, the accessed data can be dynamically allocated at runtime, e.g., using `malloc`. Fourth, many dynamic compilation systems use multiphase compilation approaches, where codes of different optimization levels are executed in different phases. For example, the Java HotSpot VM (JVM 1999) runs interpretation and JIT compilation, and several DBTs employ the two-phase translation, such as IA-32 EL (Baraz et al. 2003) and HQEMU. As the SIMD loop transformation is implemented in such systems, the execution of long-SIMD loops might start after a “warming phase” running in the short-SIMD loops. As a result, the long-SIMD loop could start at *any* iteration and the alignment of memory accesses is not determinable.

Due to the above-mentioned reasons, it is interesting to study the performance between the two alignment handling approaches—dynamic loop peeling and SIMD packing/unpacking. Figure 9 shows the SIMD packing/unpacking approach with NEON-to-AVX-256/AVX-512 transformation. Figures 9(a) and 9(b) respectively depict the flows of packing two and four short-SIMD data into a long-SIMD register; the operations of unpacking have similar flows but in a reverse direction. As shown in the figures, assuming n short-SIMD data are moved between memory and the long-SIMD register, such a movement requires n short-SIMD memory accesses and $(n - 1)$ SIMD packing/unpacking operations. Comparing with the dynamic loop peeling approach, which uses single long-SIMD memory operation, the SIMD packing/unpacking approach generates multiple instructions, not only occupying more issue slots but also causing additional data reorganization overhead (i.e., packing/unpacking).

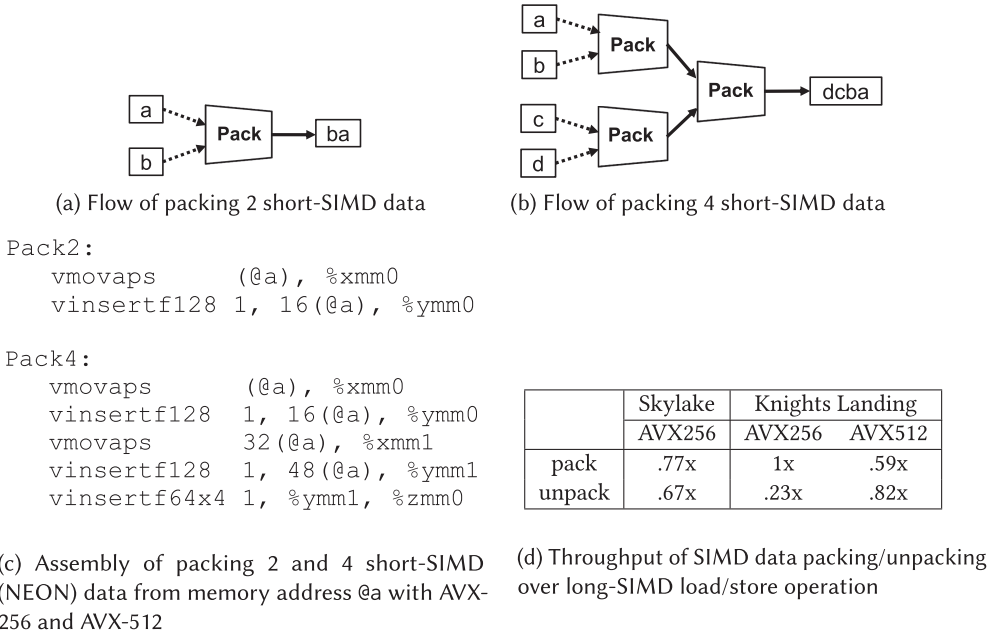


Fig. 9. Flow and throughput of the SIMD packing/unpacking approach with AVX-256 and AVX-512.

Figure 9(c) shows assembly codes of the packing operations in Figures 9(a) and 9(b), generated by the LLVM JIT compiler (assuming the short-SIMD data to be packed are contiguous in the memory). We measure the throughput of performing such SIMD packing/unpacking operations on two of the experimental platforms. The normalized throughput over that of the long-SIMD load/store operations is listed in Figure 9(d). As the results show, data movement with long-SIMD memory operations outperforms SIMD packing/unpacking. The only exception is AVX-256 packing on Knights Landing, whose performance is equal to that of the AVX-256 load operation.

Figures 10(a) and 10(b) respectively show the performance results of NEON-to-AVX-256/AVX-512 transformation on Skylake and Knights Landing. On Skylake, the SIMD packing/unpacking approach achieves an average speedup of 1.24 $\times$. In addition, dynamic loop peeling outperforms SIMD packing/unpacking for all benchmarks, achieving 1.51 $\times$ speedup on average. The results are not surprising because long-SIMD memory accesses are executed in an aligned manner after dynamic loop peeling, which have better throughput than the approach of SIMD packing/unpacking.

On Knights Landing, the SIMD packing/unpacking approach shows dramatic results compared with those on Skylake. Many benchmarks exhibit performance degradation; only a few benchmarks are slightly improved after the SIMD loop transformation. On average, the overall performance degrades by 10% and improves by 3% over the baseline when translated to AVX-256 and AVX-512, respectively. The SIMD packing/unpacking approach does not perform well on Knights Landing because the Knights Landing processor core is not as powerful as Skylake's. The Skylake processor core consists of many execution units, and one of them is dedicated for processing SIMD data reorganization operations (i.e., the Shuffle unit in port 5), allowing the processor to execute SIMD computation and packing/unpacking operations concurrently. In contrast, the Knights Landing processor core consists of fewer memory execution units. Furthermore, SIMD computation and packing/unpacking operations are processed by the same vector processing units (Intel Corporation 2016). Therefore, the performance degrades as a result of severe resource contention.

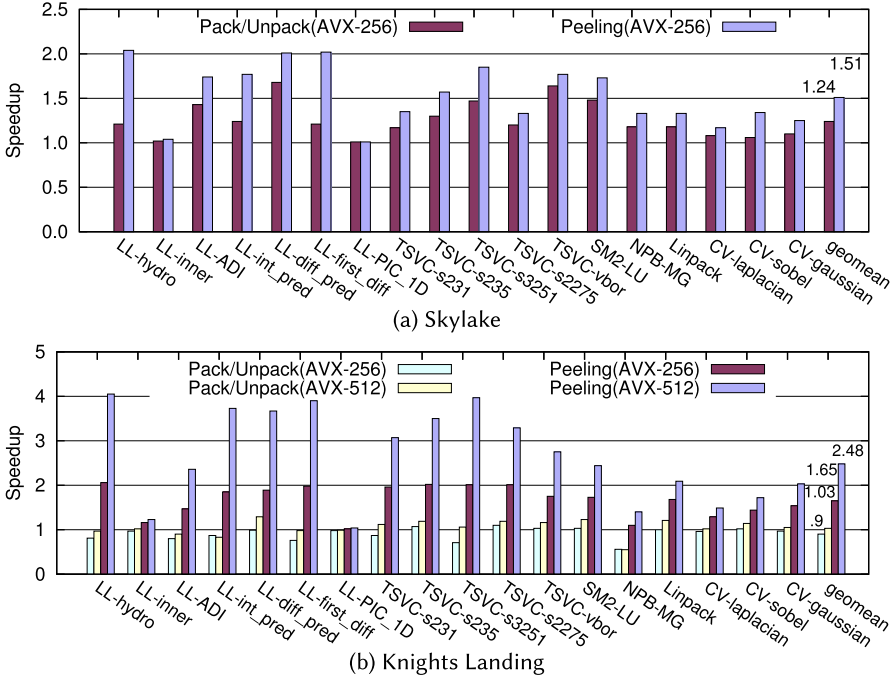


Fig. 10. Performance of SIMD packing/unpacking and dynamic loop peeling.

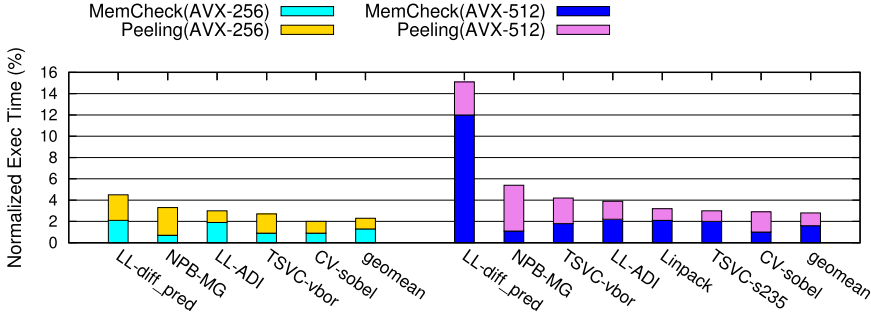


Fig. 11. Overhead of runtime memory checking and dynamic loop peeling.

The results in this section further show the strength of the dynamic loop peeling approach. Our solution imposes no extra workload on the processor and effectively improves performance across different processor architectures.

5.6 Overhead of Runtime Memory Verification and Dynamic Loop Peeling

Figure 11 shows the normalized execution time of runtime memory checking and dynamic loop peeling over the total execution time. Note that the benchmarks for which overhead is less than 2% of the total execution time are not shown in the figure. The runtime memory checking overhead is calculated as the ratio of the verification cost to the total execution time, and the overhead of dynamic loop peeling is measured as the cost of the voting algorithm.

Since the runtime memory verification is conducted only once before entering the long-SIMD loop and the execution of the long-SIMD loop is sufficiently long, the overhead is significantly amortized when transformed to AVX-256. In Figure 11, the measurement shows that runtime memory checking overhead ranges from 0% to 2% (LL-diff_pred) on Haswell and Skylake. For the transformation with AVX-512 on Knights Landing, the execution time of long-SIMD loops is reduced significantly as the parallelization factor increases to four. The results show that runtime memory verification incurs negligible overhead for most benchmarks, ranging from 0% to 2% (LL-ADI). Benchmark LL-diff_pred is the only exception that shows considerable overhead, for about 12%. On average, runtime memory checking incurs overhead of 1% and 1.6% of total execution time on AVX-256 and AVX-512, respectively.

The higher overhead in LL-diff_pred is due to a large number of memory reference pairs being checked for its loop. Column 3 of Table 1 lists the number of memory reference pairs checked at runtime for each benchmark. Most of the benchmarks have fewer than 25 checking pairs, except for LL-diff_pred and LL-ADI. These two benchmarks have lots of checking pairs because they access array elements in many different rows of a two-dimensional array; e.g., the referenced array elements in LL-diff_pred are $(a[4][i], a[5][i], a[6][i], \dots, a[13][i])$. These array elements are viewed by the static compiler as the reference pattern $(@a+4C+i, @a+5C+i, \dots, @a+13C+i)$, where $@a$ is the array base address and C is the size of one row. Since C is a large value, there exists no dependence between the array references, and the static compiler resolves that dynamic testing is not required. Our DBT could also skip runtime memory checking for this array if the same reference pattern could be retrieved. However, during the optimization phase, the static compiler maps the address of the array element in each row to one general-purpose register, e.g., mapping $@a+4C$ to register $\%r0$, $@a+5C$ to $\%r1$, and so on. This results in the reference pattern becoming $(\%r0+i, \%r1+i, \dots, \%r9+i)$, and the DBT cannot determine dependencies of these references at translation time and thus a large number of memory reference checks are produced.

Such checking overhead can be eliminated with compiler annotation. As compiler annotation is enabled, our DBT retrieves the minimum distance information of the loop, which is C for LL-diff_pred and is greater than the host SIMD vector size. Unnecessary runtime memory checking is eliminated, and the performance of LL-diff_pred is improved by 12%. With the proposed compiler annotation approach, the overall performance is further improved by a factor of 1% and 1.6% on AVX-256 and AVX-512, respectively.

For dynamic loop peeling, since the voting algorithm is conducted only once and the execution of the long-SIMD loop is sufficiently long, the overhead of dynamic loop peeling is also mostly amortized. As the measurement in Figure 11 shows, dynamic loop peeling incurs less than 2% overhead for most benchmarks. The only significant overhead is observed in benchmark NPB-MG on AVX-512, for about 4% of the total execution time.

5.7 Performance Results Compared with Native Execution

Figure 12 shows the performance of ARM NEON-to-x86 AVX-256 translation compared with native execution on the Skylake machine. All benchmarks are vectorized and compiled to native binary using GCC with the flags “-O3 -fno-unroll-loops -ftree-vectorized -ffast-math -mavx2 -mfpmath=sse.” As Figure 12 shows, the performance of cross-ISA SIMD loop transformation is 1.6× slower than the native one on average. Such slowdown is due to several factors that impede the DBT performance, such as translation overhead and emulation overhead (e.g., handling of indirect branches, context switches between DBT dispatcher and code cache, and helper function calls). In the cross-ISA HQEMU DBT, loads/stores of the emulated guest registers also incur extra emulation overhead when the execution transitions among translated code fragments within the code cache. In Figure 12, the performance of benchmark LL-PIC_1D and CV-laplacian is worse

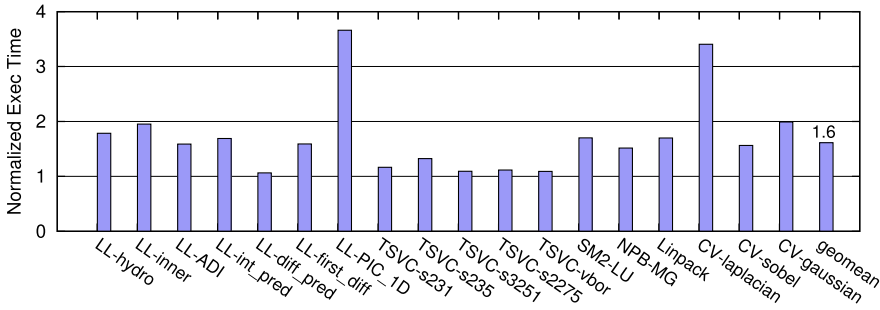


Fig. 12. Performance results of NEON-to-AVX-256 translation over native execution on Skylake.

than the other benchmarks. The reason is that helper functions, which are used to emulate complex ARM instructions, are invoked frequently in the non-SIMD part of these two benchmarks. Their overhead lengthens the total DBT execution time and thus causes a performance drop compared with the native execution.

6 RELATED WORK

Vectorization and dynamic compilation. SIMD architectures achieve power and performance efficiency by exploiting data-level parallelism. Many auto-vectorization techniques have been proposed in the literature to exploit SIMD parallelism, such as loop-based vectorization (Bik et al. 2002; Karrenberg and Hack 2011; Naishlos 2004), SLP-based vectorization (Larsen and Amarasinghe 2000; Liu et al. 2012; Porpodas and Jones 2015; Porpodas et al. 2015; Shin et al. 2003; Zhou and Xue 2016a), and a combination of the above two techniques (Zhou and Xue 2016b). However, these efforts mainly focus on source-level approaches, which are widely used in static compilers. These approaches still need to recompile applications to leverage new hardware benefits, such as longer SIMD, additional registers, and new instructions.

Nuzman et al. (2011) proposed a split compilation framework, where the vectorization process is split into two phases. The offline compiler conducts auto-vectorization of the machine-independent part to IR in the first phase, and the JIT compiler in the second phase resolves the final vectorization based on the host vector parameters. The offline compiler in their system targets source programs and conducts typical scalar-to-vector transformation. Barik et al. (2010) extended the Jikes RVM dynamic compiler, which explores vectorization opportunities from scalars by using dynamic programming. Pajuelo et al. (2002) proposed a DBT system that conducts auto-vectorization from scalars to vectors. Their system requires new hardware support to speculatively execute vectorized code and fall back to scalar code when an invalid vectorization is detected by the hardware. In contrast to these works, our DBT conducts translation from short SIMD to long SIMD, which is a pure software solution.

The most closely related works are Hallou et al. (2015, 2017), which also use a dynamic binary rewriting technique to convert short-SIMD binaries to long-SIMD codes. Their work is implemented in a same-ISA DBT for the x86 architecture. When a short-SIMD memory operation could become misaligned in a long-SIMD form, their DBT system does not emit a long-SIMD memory instruction but uses multiple short-SIMD memory accesses and SIMD packing/unpacking instructions. In contrast, our approach is designed in a retargetable DBT that supports cross-ISA translation. Our new contributions are that we propose using (1) a general machine-independent IR layer for SIMD loop transformation and optimization and (2) a general approach of dynamic loop peeling to enforce the alignment of long-SIMD memory accesses. Our work is more portable than

a same-ISA DBT, and our alignment handling approach can achieve portable performance across different SIMD ISAs.

Auto-parallelization. Kotha et al. (2010) proposed a static binary translation framework that decompiles the binary to an IR layer and looks for thread-level parallelism in the IR level. In contrast, we conduct SIMD loop transformation in a DBT that can handle code discovery problems and self-modifying codes that are hard to deal with by static binary translators. The DAISY system (Ebcioglu and Altman 1997) and Transmeta CMS (Dehnert et al. 2003) achieve instruction-level parallelism by dynamically translating scalar codes into VLIW codes. The code generation in DAISY must guarantee that no dependencies exist between operations in a VLIW instruction at the time of translation, while the Transmeta processor provides a special alias hardware (Klaiber 2000) that allows CMS to reorder selected memory instructions, with hardware taking on the burden of verifying dependencies. Unlike their systems, our DBT improves parallelism with vectorization. The required translation techniques are quite different. In addition, we use a conditional translation approach in which runtime checking for memory references is conducted if dependencies cannot be verified at translation time. Our software-based approach forces the execution of the original short-SIMD loop when any dependence is detected in the first place.

Compiler annotation. Using compiler annotation to assist binary translation is not a new technique. Wang (2003) proposed that an optimizing compiler can provide useful information, such as function boundaries, split functions, jump table information, function addresses, and code labels, for the binary translator, allowing it to use non-heuristic approaches to translate binary code. Xu et al. (2007) proposed annotating non-volatile and thread-local variables that are spilled into the stack due to insufficient guest physical registers. Memory accesses of these variables are transformed to register accesses if the host has enough registers. Williams et al. (2009) proposed a meta-data management service that stores compiler-level information in an XML database or a compact binary format. The information of virtual function table is exploited to improve indirect branch handling in their DBT system. Our DBT is assisted with the dependence distance information to optimize memory dependence verification.

7 CONCLUSION

In this article, we present a dynamic binary translation technique of short-SIMD to long-SIMD loop transformation. We propose a general approach that translates short-SIMD binaries to machine-independent IR, conducts SIMD loop transformation at this IR level, and finally translates to long-SIMD instructions. Memory dependence is a critical issue for the correctness of vectorization. Such an issue is often determined at compile time by static compilers since the semantic information for dependence analysis can often be resolved from the high-level programming language. However, designing memory dependence analysis is more challenging for DBTs because DBTs deal with binaries where high-level information may be lost. To solve this issue, we present runtime memory checking as a general solution and utilize compiler annotation to eliminate runtime checking overhead. We also identify the alignment problem where memory accesses aligned in the original binary can become misaligned after SIMD loop transformation. The code duplication problem caused by DBT can further exacerbate the misalignment. The proposed approach of changing DBT termination conditions relieves such misalignment, and a general solution using dynamic loop peeling is also presented. We demonstrate that the SIMD loop transformation can significantly improve the performance of short-SIMD binaries, not only from better parallelism of the long-SIMD architectures but also from exploiting new SIMD hardware features. The proposed concept can also be applied to same-ISA DBTs, static binary translators, and other JIT systems.

REFERENCES

- Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. 2000. Dynamo: A transparent dynamic optimization system. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*. 1–12.
- Utpal K. Banerjee. 1976. *Data Dependence in Ordinary Programs*. Technical Report.
- Leonid Baraz, Tevi Devor, Orna Etzion, Shalom Goldenberg, Alex Skaletsky, Yun Wang, and Yigal Zemach. 2003. IA-32 execution layer: A two-phase dynamic translator designed to support IA-32 applications on Itanium-based systems. In *IEEE/ACM International Symposium on Microarchitecture*.
- Rajkishore Barik, Jisheng Zhao, and Vivek Sarkar. 2010. Efficient selection of vector instructions using dynamic programming. In *Annual IEEE/ACM International Symposium on Microarchitecture*. 201–212.
- Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *USENIX Annual Technical Conference*. 41–46.
- Aart J. C. Bik, Milind Girkar, Paul M. Grey, and Xinmin Tian. 2002. Automatic intra-register vectorization for the Intel architecture. *International Journal of Parallel Programming* 30, 2 (2002), 65–98.
- Derek Bruening, Timothy Garnett, and Saman Amarasinghe. 2003. An infrastructure for adaptive dynamic optimization. In *International Symposium on Code Generation and Optimization*. 265–275.
- Patricio Bulic and Veselko Gustin. 2005. On dependence analysis for SIMD enhanced processors. In *International Conference on High Performance Computing for Computational Science*. 527–540.
- James C. Dehnert, Brian K. Grant, John P. Banning, Richard Johnson, Thomas Kistler, Alexander Klaiber, and Jim Mattson. 2003. The transmeta code morphing software: Using speculation, recovery, and adaptive retranslation to address real-life challenges. In *International Symposium on Code Generation and Optimization*. 15–24.
- Evelyn Duesterwald and Vasanth Bala. 2000. Software profiling for hot path prediction: Less is more. In *International Conference on Architectural Support for Programming Languages and Operating Systems*. 202–211.
- Kemal Ebcioglu and Erik R. Altman. 1997. DAISY: Dynamic compilation for 100% architectural compatibility. In *International Symposium on Computer Architecture*. 26–37.
- Sheng-Yu Fu, Ding-Yong Hong, Jan-Jan Wu, Pangfeng Liu, and Wei-Chung Hsu. 2015. SIMD code translation in an enhanced HQEMU. In *IEEE International Conference on Parallel and Distributed Systems*. 507–514.
- Nabil Hallou, Erven Rohou, and Philippe Clauss. 2017. Runtime vectorization transformations of binary code. *International Journal of Parallel Programming* 45, 6 (2017), 1536–1565.
- Nabil Hallou, Erven Rohou, Philippe Clauss, and Alain Ketterlin. 2015. Dynamic re-vectorization of binary code. In *International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation*. 228–237.
- Ding-Yong Hong, Sheng-Yu Fu, Yu-Ping Liu, Jan-Jan Wu, and Wei-Chung Hsu. 2016. Exploiting longer SIMD lanes in dynamic binary translation. In *IEEE International Conference on Parallel and Distributed Systems*. 853–860.
- Ding-Yong Hong, Chun-Chen Hsu, Pen-Chung Yew, Jan-Jan Wu, Wei-Chung Hsu, Yeh-Ching Chung, Pangfeng Liu, and Chien-Min Wang. 2012. HQEMU: A multi-threaded and retargetable dynamic binary translator on multicores. In *Symposium on Code Generation and Optimization*. 104–113.
- Intel Corporation. 2016. Intel®64 and IA-32 Architectures Optimization Reference Manual.
- JVM. 1999. HotSpot parallel collector. In *Memory Management in the Java HotSpot Virtual Machine Whitepaper*.
- Ralf Karrenberg and Sebastian Hack. 2011. Whole-function vectorization. In *International Symposium on Code Generation and Optimization*. 141–150.
- Vladimir Kiriansky, Derek Bruening, and Saman P. Amarasinghe. 2002. Secure execution via program shepherding. In *Security Symposium*. 191–206.
- Alexander Klaiber. 2000. *The Technology Behind the Crusoe Processors*. Technical Report.
- Xiangyun Kong, David Klappholz, and Kleanthis Psarris. 1991. The I test: An improved dependence test for automatic parallelization and vectorization. *IEEE Transactions on Parallel and Distributed Systems* 2, 3 (1991), 342–349.
- Aparna Kotha, Kapil Anand, Matthew Smithson, Greeshma Yellareddy, and Rajeev Barua. 2010. Automatic parallelization in a binary rewriter. In *IEEE/ACM International Symposium on Microarchitecture*. 547–557.
- Samuel Larsen and Saman Amarasinghe. 2000. Exploiting superword level parallelism with multimedia instruction sets. In *ACM Conference on Programming Language Design and Implementation*. 145–156.
- Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization*. 75–88.
- Jianhui Li, Qi Zhang, Shu Xu, and Bo Huang. 2006. Optimizing dynamic binary translation for SIMD instructions. In *International Symposium on Code Generation and Optimization*. 269–280.
- Jun Liu, Yuanrui Zhang, Ohyoung Jang, Wei Ding, and Mahmut Kandemir. 2012. A compiler framework for extracting superword level parallelism. In *ACM Conference on Programming Language Design and Implementation*. 347–358.
- Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: Building customized program analysis tools with dynamic instrumentation. In *ACM Conference on Programming Language Design and Implementation*. 190–200.

- Luc Michel, Nicolas Fournel, and Frederic Petrot. 2011. Speeding-up SIMD instructions dynamic binary translation in embedded processor simulation. In *Design, Automation & Test in Europe Conference & Exhibition*. 1530–1591.
- Dorit Naishlos. 2004. Auto-vectorization in GCC. In *Proceedings of the GCC Developers Summit*. 105–117.
- Nicholas Nethercote and Julian Seward. 2007. Valgrind: A framework for heavyweight dynamic binary instrumentation. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*. 89–100.
- Dorit Nuzman, Sergei Dyshel, Erven Rohou, Ira Rosen, Kevin Williams, David Yuste, Albert Cohen, and Ayal Zaks. 2011. Vapor SIMD: Auto-vectorize once, run everywhere. In *International Symposium on Code Generation and Optimization*. 151–160.
- Alex Pajuelo, Antonio Gonzalez, and Mateo Valero. 2002. Speculative dynamic vectorization. In *International Symposium on Computer Architecture*. 271–280.
- Vasileios Porpodas and Timothy M. Jones. 2015. Throttling automatic vectorization: When less is more. In *International Conference on Parallel Architecture and Compilation Techniques*. 432–444.
- Vasileios Porpodas, Alberto Magni, and Timothy M. Jones. 2015. PSLP: Padded SLP automatic vectorization. In *International Symposium on Code Generation and Optimization*. 190–201.
- Kevin Scott and Jack Davidson. 2001. *Strata: A Software Dynamic Translation Infrastructure*. Technical Report. Charlottesville, VA.
- Jaewook Shin, Jacqueline Chame, and Mary W. Hall. 2003. Exploiting superword-level locality in multimedia extension architectures. *Journal of Instruction-Level Parallelism* 5 (2003), 1–28.
- Cheng Wang, Shiliang Hu, Ho-seop Kim, Sreekumar R. Nair, Mauricio Breternitz, Zhiwei Ying, and Youfeng Wu. 2007. StarDBT: An efficient multi-platform dynamic binary translation system. In *Asia-Pacific Conference on Advances in Computer Systems Architecture*. 4–15.
- Fu-Hwa Wang. 2003. Compiler annotation for binary translation tools. May 8, 2003. U.S. Patent 20030088860 A1.
- Daniel Williams, Jason D. Hiser, and Jack W. Davidson. 2009. Using program metadata to support SDT in object-oriented applications. In *Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems*. 55–62.
- Michael Wolfe and Chau-Wen Tseng. 1992. The power test for data dependence. *IEEE Transactions on Parallel and Distributed Systems* 3, 5 (1992), 591–601.
- Chaohao Xu, Jianhui Li, Tao Bao, Yun Wang, and Bo Huang. 2007. Metadata driven memory optimizations in dynamic binary translator. In *International Conference on Virtual Execution Environments*. 148–157.
- Matt T. Yourst. 2007. PTLsim: A cycle accurate full system x86-64 microarchitectural simulator. In *International Symposium on Performance Analysis of Systems & Software*. 23–34.
- Hao Zhou and Jingling Xue. 2016a. A compiler approach for exploiting partial SIMD parallelism. *ACM Transactions on Architecture and Code Optimization* 13, 1 (2016), 11:1–11:26.
- Hao Zhou and Jingling Xue. 2016b. Exploiting mixed SIMD parallelism by reducing data reorganization overhead. In *International Symposium on Code Generation and Optimization*. 59–69.
- Hans Zima and Barbara Chapman. 1991. *Supercompilers for Parallel and Vector Computers*. ACM, New York.

Received April 2017; revised September 2017; accepted December 2017